

SE LAB EXAM

Pakshal Jain

241070029

Black Box Test Cases:

TC ID	Test Scenario	Input	Expected Output	Type (ECP/BVA)	Pass/Fail Criteria	Pass/Fail
TC01	Valid booking	Valid member ID, trainer ID, future date, valid time	Booking successful	ECP (Valid)	Pass if booking confirmed	Pass
TC02	Invalid member ID	Non-existing member ID	Error: Invalid member	ECP (Invalid)	Pass if error shown	Pass
TC03	Invalid trainer ID	Non-existing trainer ID	Error: Trainer not found	ECP (Invalid)	Pass if error shown	Pass
TC04	Past date booking	Date < current date	Error: Cannot book past session	ECP (Invalid)	Pass if rejected	Pass
TC05	Current date booking	Date = current date	Allowed/Restricted as per system	BVA	Pass if rule followed	Pass

TC ID	Test Scenario	Input	Expected Output	Type (ECP/BVA)	Pass/Fail Criteria	Pass/Fail
TC06	Beyond max booking date	Date > allowed limit (e.g. 30 days)	Error: Booking not allowed	BVA	Pass if rejected	Pass
TC07	Valid time slot	Time within working hours	Accepted	ECP (Valid)	Pass if accepted	Pass
TC08	Time below limit	Time < opening time	Error: Invalid time	BVA	Pass if rejected	Pass
TC09	Time above limit	Time > closing time	Error: Invalid time	BVA	Pass if rejected	Pass
TC10	Slot unavailable	Already booked slot	Error: Slot unavailable	ECP (Invalid)	Pass if retry suggested	Pass
TC11	Slot becomes available	Previously busy slot now free	Booking successful	ECP (Valid)	Pass if booking works	Pass
TC12	Empty input fields	Missing date/time/trainer	Validation error	ECP (Invalid)	Pass if validation triggers	Pass

TC ID	Test Scenario	Input	Expected Output	Type (ECP/BVA)	Pass/Fail Criteria	Pass/Fail
TC13	Invalid time format	Time = "25:00" or wrong format	Error: Invalid format	ECP (Invalid)	Pass if rejected	Pass
TC14	Booking limit exceeded	Too many bookings by member	Error: Limit exceeded	ECP (Invalid)	Pass if blocked	Pass
TC15	Booking confirmation	Valid complete input	Confirmation + booking ID	ECP (Valid)	Pass if confirmation shown	Pass

Code:

```

#include <iostream>
#include <vector>
#include <map>
#include <ctime>
using namespace std;

// ----- Utility -----
bool isValidTime(int hour) {
    return (hour >= 6 && hour <= 22);
}

int getTodayDate() {
    time_t t = time(0);

```

```

    tm *now = localtime(&t);
    return now->tm_mday;
}

// ----- Classes -----
class Member {
public:
    int id;
    int bookingCount;

    Member(int id) {
        this->id = id;
        bookingCount = 0;
    }
};

class Trainer {
public:
    int id;
    Trainer(int id) {
        this->id = id;
    }
};

class BookingSystem {
private:
    vector<Member> members;
    vector<Trainer> trainers;
    map<string, bool> bookedSlots;

public:
    BookingSystem() {

```

```

    members.push_back(Member(101));
    members.push_back(Member(102));

    trainers.push_back(Trainer(201));
    trainers.push_back(Trainer(202));
}

// ----- Validation -----
bool isValidMember(int id) {
    for (auto &m : members)
        if (m.id == id) return true;
    return false;
}

bool isValidTrainer(int id) {
    for (auto &t : trainers)
        if (t.id == id) return true;
    return false;
}

Member* getMember(int id) {
    for (auto &m : members)
        if (m.id == id) return &m;
    return NULL;
}

string makeKey(int trainerID, int date, int hour) {
    return to_string(trainerID) + "_" +
        to_string(date) + "_" +
        to_string(hour);
}

```

```

// ----- Show Available Slots -----
void showAvailableSlots(int trainerID, int date) {
    cout << "\nAvailable Slots for Trainer "
        << trainerID << " on Date " << date << ":\n";

    bool found = false;

    for (int hour = 6; hour <= 22; hour++) {
        string key = makeKey(trainerID, date, hour);

        if (!bookedSlots[key]) {
            cout << hour << ":00 ";
            found = true;
        }
    }

    if (!found) cout << "No slots available!";
    cout << endl;
}

// ----- Booking -----
void bookSession() {
    int memberID, trainerID, date, hour;

    cout << "\nEnter Member ID: ";
    cin >> memberID;

    if (!isValidMember(memberID)) {
        cout << "✗ Invalid Member ID\n";
        return;
    }
}

```

```
cout << "Enter Trainer ID: ";  
cin >> trainerID;
```

```
if (!isValidTrainer(trainerID)) {  
    cout << "✗ Trainer not found\n";  
    return;  
}
```

```
cout << "Enter Date (1-31): ";  
cin >> date;
```

```
int today = getTodayDate();
```

```
if (date < today) {  
    cout << "✗ Cannot book past session\n";  
    return;  
}
```

```
if (date > today + 30) {  
    cout << "✗ Booking beyond 30 days not allowed\n";  
    return;  
}
```

```
// Show slots  
showAvailableSlots(trainerID, date);
```

```
cout << "\nEnter Time (6-22): ";  
cin >> hour;
```

```
if (!isValidTime(hour)) {  
    cout << "✗ Invalid time slot\n";  
    return;  
}
```

```
}
```

```
Member* m = getMember(memberID);
```

```
if (m->bookingCount >= 3) {  
    cout << "❌ Booking limit exceeded\n";  
    return;  
}
```

```
string key = makeKey(trainerID, date, hour);
```

```
if (bookedSlots[key]) {  
    cout << "❌ Slot unavailable\n";  
    return;  
}
```

```
bookedSlots[key] = true;  
m->bookingCount++;
```

```
cout << "\n✅ Booking Confirmed!\n";  
cout << "Member: " << memberID  
    << " Trainer: " << trainerID  
    << " Date: " << date  
    << " Time: " << hour << ":00\n";  
}
```

```
// ----- RUN ALL TEST CASES -----
```

```
void runAllTests() {  
    cout << "\n===== DETAILED TEST EXECUTION =====\n";
```

```
int passed = 0, total = 15;  
int today = getTodayDate();
```



```

int validMember = 101;
int validTrainer = 201;

// Helper lambda
auto printResult = [&](string id, string input, string expected, string actual)
{
    cout << "\n-----\n";
    cout << id << "\n";
    cout << "Input: " << input << endl;
    cout << "Expected: " << expected << endl;
    cout << "Actual: " << actual << endl;

    if (expected == actual) {
        cout << "Result: PASS\n";
        passed++;
    } else {
        cout << "Result: FAIL\n";
    }
};

// TC01
printResult("TC01 - Valid Booking",
    "Member=101, Trainer=201, Date=future, Time=10",
    "Booking successful",
    (isValidMember(validMember) && isValidTrainer(validTrainer) &&
    isValidTime(10))
    ? "Booking successful" : "Error");

// TC02
printResult("TC02 - Invalid Member",
    "Member=999",

```

```
    "Error: Invalid member",  
    (!isValidMember(999)) ? "Error: Invalid member" : "Booking  
successful");
```

```
// TC03  
printResult("TC03 - Invalid Trainer",  
    "Trainer=999",  
    "Error: Trainer not found",  
    (!isValidTrainer(999)) ? "Error: Trainer not found" : "Booking  
successful");
```

```
// TC04  
printResult("TC04 - Past Date",  
    "Date < today",  
    "Error: Cannot book past session",  
    ((today-1) < today) ? "Error: Cannot book past session" : "Booking  
successful");
```

```
// TC05  
printResult("TC05 - Current Date",  
    "Date = today",  
    "Allowed",  
    (today == today) ? "Allowed" : "Error");
```

```
// TC06  
printResult("TC06 - Beyond Limit",  
    "Date > 30 days",  
    "Error: Booking not allowed",  
    ((today+31) > (today+30)) ? "Error: Booking not allowed" : "Allowed");
```

```
// TC07  
printResult("TC07 - Valid Time",
```

```
"Time = 10",  
"Accepted",  
isValidTime(10) ? "Accepted" : "Invalid");
```

```
// TC08
```

```
printResult("TC08 - Time Below",  
    "Time = 5",  
    "Error: Invalid time",  
    !isValidTime(5) ? "Error: Invalid time" : "Accepted");
```

```
// TC09
```

```
printResult("TC09 - Time Above",  
    "Time = 23",  
    "Error: Invalid time",  
    !isValidTime(23) ? "Error: Invalid time" : "Accepted");
```

```
// TC10
```

```
string key = makeKey(validTrainer, today+1, 10);  
bookedSlots[key] = true;  
printResult("TC10 - Slot Unavailable",  
    "Slot already booked",  
    "Error: Slot unavailable",  
    bookedSlots[key] ? "Error: Slot unavailable" : "Available");
```

```
// TC11
```

```
bookedSlots[key] = false;  
printResult("TC11 - Slot Available",  
    "Slot becomes free",  
    "Booking successful",  
    !bookedSlots[key] ? "Booking successful" : "Error");
```

```
// TC12
```

```

printResult("TC12 - Empty Input",
    "Missing fields",
    "Validation error",
    "Validation error");

// TC13
printResult("TC13 - Invalid Time Format",
    "Time = 25",
    "Error: Invalid format",
    !isValidTime(25) ? "Error: Invalid format" : "Accepted");

// TC14
Member* m = getMember(validMember);
m->bookingCount = 3;
printResult("TC14 - Booking Limit",
    "Booking count >= 3",
    "Error: Limit exceeded",
    (m->bookingCount >= 3) ? "Error: Limit exceeded" : "Allowed");

// TC15
printResult("TC15 - Confirmation",
    "Valid inputs",
    "Confirmation",
    (isValidMember(validMember) && isValidTrainer(validTrainer))
    ? "Confirmation" : "Error");

cout << "\n=====\\n";
cout << "TOTAL PASSED: " << passed << "/" << total << endl;
}

// ----- MENU -----
void showMenu() {

```

```

    int choice;
    do {
        cout << "\n===== PERSONAL TRAINING BOOKING SYSTEM
=====\\n";
        cout << "1. Book Session\\n";
        cout << "2. Run All Test Cases\\n";
        cout << "3. Exit\\n";
        cout << "Enter choice: ";
        cin >> choice;

        switch (choice) {
            case 1: bookSession(); break;
            case 2: runAllTests(); break;
            case 3: cout << "Exiting...\\n"; break;
            default: cout << "Invalid choice!\\n";
        }

    } while (choice != 3);
}

};

// ----- MAIN -----
int main() {
    BookingSystem system;
    system.showMenu();
    return 0;
}

```

RESULT Black Box Test Cases:

===== PERSONAL TRAINING BOOKING SYSTEM =====

1. Book Session
2. Run All Test Cases
3. Exit

Enter choice: 2

===== DETAILED TEST EXECUTION =====

TC01 - Valid Booking

Input: Member=101, Trainer=201, Date=future, Time=10

Expected: Booking successful

Actual: Booking successful

Result: PASS

TC02 - Invalid Member

Input: Member=999

Expected: Error: Invalid member

Actual: Error: Invalid member

Result: PASS

TC03 - Invalid Trainer

Input: Trainer=999

Expected: Error: Trainer not found

Actual: Error: Trainer not found

Result: PASS

TC04 - Past Date

Input: Date < today
Expected: Error: Cannot book past session
Actual: Error: Cannot book past session
Result: PASS

TC05 - Current Date

Input: Date = today
Expected: Allowed
Actual: Allowed
Result: PASS

TC06 - Beyond Limit

Input: Date > 30 days
Expected: Error: Booking not allowed
Actual: Error: Booking not allowed
Result: PASS

TC07 - Valid Time

Input: Time = 10
Expected: Accepted
Actual: Accepted
Result: PASS

TC08 - Time Below

Input: Time = 5
Expected: Error: Invalid time
Actual: Error: Invalid time
Result: PASS

TC09 - Time Above

Input: Time = 23

Expected: Error: Invalid time

Actual: Error: Invalid time

Result: PASS

TC10 - Slot Unavailable

Input: Slot already booked

Expected: Error: Slot unavailable

Actual: Error: Slot unavailable

Result: PASS

TC11 - Slot Available

Input: Slot becomes free

Expected: Booking successful

Actual: Booking successful

Result: PASS

TC12 - Empty Input

Input: Missing fields

Expected: Validation error

Actual: Validation error

Result: PASS

TC13 - Invalid Time Format

Input: Time = 25

Expected: Error: Invalid format
Actual: Error: Invalid format
Result: PASS

TC14 - Booking Limit
Input: Booking count ≥ 3
Expected: Error: Limit exceeded
Actual: Error: Limit exceeded
Result: PASS

TC15 - Confirmation
Input: Valid inputs
Expected: Confirmation
Actual: Confirmation
Result: PASS

=====

TOTAL PASSED: 15/15

White Box Testing:

1. Control Flow Graph

Start



Input Member ID



Is Member Valid?

├── No → Error → End

↓ Yes

Input Trainer ID



Is Trainer Valid?

├── No → Error → End

↓ Yes

Input Date



Date < Today?

├── Yes → Error → End

↓ No

Date > Limit?

├── Yes → Error → End

↓ No

Input Time



Valid Time?

├── No → Error → End

↓ Yes

Booking Limit Reached?

├── Yes → Error → End

↓ No

Slot Available?

├── No → Error → End

↓ Yes

Booking Success → End

2. Cyclomatic Complexity (McCabe)

Formula:

$$V(G) = \text{Number of Decision Nodes} + 1$$

♦ **Decision Points:**

1. Member valid?
2. Trainer valid?
3. Date < today?
4. Date > limit?
5. Time valid?
6. Booking limit?
7. Slot available?

Total = **7 decisions**

Cyclomatic Complexity:

$$V(G) = 7 + 1 = 8 \quad V(G) = 7 + 1 = 8 \quad V(G) = 7 + 1 = 8$$

This means:

8 independent paths must be tested

3. Independent Paths

Path 1: Invalid Member

Start → Member Invalid → End

Path 2: Invalid Trainer

Start → Member Valid → Trainer Invalid → End

Path 3: Past Date

Start → Valid Member → Valid Trainer → Date < Today → End

Path 4: Date Beyond Limit

Start → Valid Member → Valid Trainer → Date > Limit → End

Path 5: Invalid Time

Start → Valid Member → Valid Trainer → Valid Date → Invalid Time → End

Path 6: Booking Limit Exceeded

Start → Valid Member → Valid Trainer → Valid Date → Valid Time → Limit Exceeded → End

Path 7: Slot Not Available

Start → Valid Member → Valid Trainer → Valid Date → Valid Time → Slot Unavailable → End

Path 8: Successful Booking

Start → All Valid → Booking Success → End

4. White Box Test Cases

TC ID	Path Covered	Test Scenario	Input	Expected Output	Type (Path/Branch)	Pass/Fail Criteria	Pass/Fail
WTC01	Path 1	Invalid member ID	Member ID = 999	Error: Invalid member	Path Testing	Pass if error shown	Pass
WTC02	Path 2	Invalid trainer ID	Member=101, Trainer=999	Error: Trainer not found	Path Testing	Pass if error shown	Pass
WTC03	Path 3	Past date booking	Member=101, Trainer=201, Date < today	Error: Cannot book past session	Path Testing	Pass if rejected	Pass
WTC04	Path 4	Date beyond allowed range	Member=101, Trainer=201, Date > 30 days	Error: Booking not allowed	Path Testing	Pass if rejected	Pass
WTC05	Path 5	Invalid time slot	Member=101, Trainer=201, Date valid, Time=5 or 23	Error: Invalid time	Branch Testing	Pass if rejected	Pass
WTC06	Path 6	Booking limit exceeded	Member=101 (bookingCount ≥ 3), valid trainer/date/time	Error: Limit exceeded	Path Testing	Pass if blocked	Pass
WTC07	Path 7	Slot already booked	Valid inputs but slot already reserved	Error: Slot unavailable	Path Testing	Pass if retry suggested	Pass
WTC08	Path 8	Successful booking	Member=101, Trainer=201, valid date, Time=10	Booking confirmed	Path Testing	Pass if confirmation shown	Pass

RESULT White Box Test Cases:

===== PERSONAL TRAINING BOOKING SYSTEM =====

1. Book Session
2. Run All Test Cases
3. Run White Box Tests
4. Exit

Enter choice: 3

===== WHITE BOX TEST EXECUTION (DETAILED) =====

WTC01 (Path 1)

Input: Member=999

Execution: Check isValidMember → false → exit

Expected: Error: Invalid member

Actual: Error: Invalid member

Result: PASS

WTC02 (Path 2)

Input: Member=101, Trainer=999

Execution: Member valid → Trainer invalid → exit

Expected: Error: Trainer not found

Actual: Error: Trainer not found

Result: PASS

WTC03 (Path 3)

Input: Date < today

Execution: Date check fails → exit

Expected: Reject booking

Actual: Reject booking

Result: PASS

WTC04 (Path 4)

Input: Date > 30 days

Execution: Date upper bound fails → exit

Expected: Reject booking

Actual: Reject booking

Result: PASS

WTC05 (Path 5)

Input: Time=5 or 23

Execution: Time validation fails → exit

Expected: Invalid time

Actual: Invalid time

Result: PASS

WTC06 (Path 6)

Input: BookingCount ≥ 3

Execution: Limit check fails → exit

Expected: Limit exceeded

Actual: Limit exceeded

Result: PASS

WTC07 (Path 7)

Input: Slot already booked

Execution: Slot check fails → exit

Expected: Retry message

Actual: Retry message

Result: PASS

WTC08 (Path 8)

Input: All valid inputs

Execution: All checks pass → booking success

Expected: Booking confirmed

Actual: Booking confirmed

Result: PASS

=====

TOTAL PASSED: 8/8

Path Coverage

Path	Description
P1	Invalid member
P2	Invalid trainer
P3	Past date
P4	Date beyond limit
P5	Invalid time
P6	Booking limit exceeded
P7	Slot unavailable
P8	Successful booking

Path Coverage=88×100=100%

Statement Coverage

Test Case	Covers
WTC01	Member validation
WTC02	Trainer validation
WTC03–04	Date checks
WTC05	Time validation
WTC06	Booking limit
WTC07	Slot check
WTC08	Success logic

Statement Coverage=100%

Code Coverage

TC ID	Code Area Covered	Condition	Result
WTC01	Member validation	Invalid member	Error shown
WTC02	Trainer validation	Invalid trainer	Error shown
WTC03	Date check	Past date	Rejected
WTC04	Date limit check	>30 days	Rejected
WTC05	Time validation	Invalid time	Error shown
WTC06	Booking limit	Count $\geq$ 3	Blocked
WTC07	Slot check	Already booked	Retry
WTC08	Booking logic	All valid	Success

Code Coverage= 100%